AGENT EVOLUTION · NOTEBOOK 1 OF 8

Version 1 — Prompt Anatomy: An Agent with a Brain, but No Hands

A guide to `prompt_anatomy_agent_brain_no_hands.ipynb`: what the agent is, how its prompt is built, what `create_agent()` creates, and what happens inside one call.

PA

Tutor: Prudhvi Akella
Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Stack: LangChain `create_agent` · OpenAI chat model Tools: none (`tools=[]`)

1. Where Version 1 sits
2. The end-to-end workflow
3. Brain and hands
4. Anatomy of the system prompt
5. What `create_agent()` builds
6. The agent loop
7. One call, step by step
8. Reading the answer
9. The five test questions
10. Why the constraints exist
11. The limitation, and what Version 2 changes
12. Key takeaways

1 Where Version 1 sits

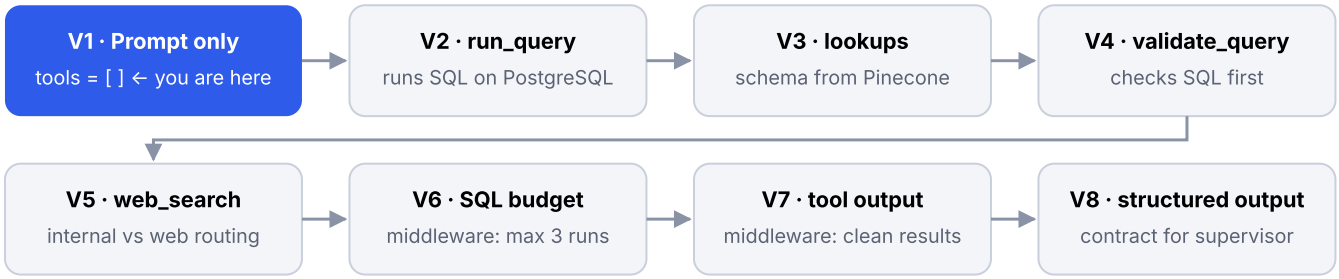


Figure 1. The series. Every version keeps the same agent and adds one capability.

All eight notebooks build one agent: an AI Data Analyst for an e-commerce marketplace. It turns business questions into SQL.

Version 1 is the starting point. The agent has a model and a system prompt, and nothing else. It can read a question and propose SQL. It cannot run the SQL or see any data.

This version teaches two things: how to build a prompt from clear parts, and what an agent is before tools are added.

2 The end-to-end workflow

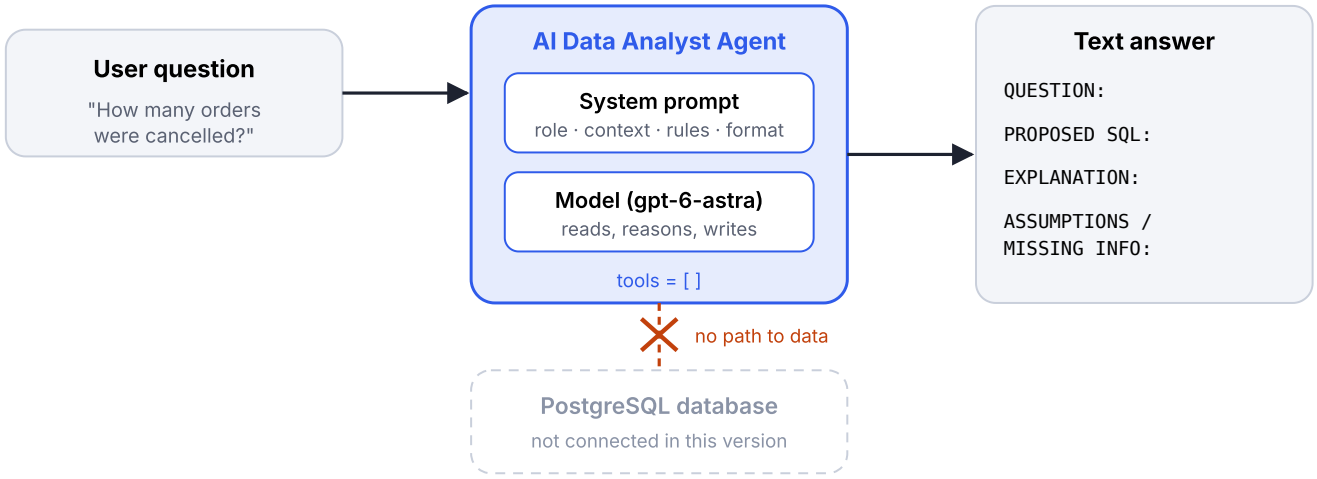


Figure 2. Version 1 end to end. The question goes in, text comes out. Nothing reaches the database.

1. **The user asks a business question.** Example: “How many orders were cancelled?”
2. **The agent gives the model the system prompt plus the question.** The prompt sets the role, the known table names, the rules and the output format.
3. **The model writes one text answer** with four labelled sections. The SQL in it is a *proposal*. Nothing runs it.
4. **The answer comes back to your code.** The notebook prints `result["messages"][-1].content`.

No validation, database, retrieval or web search happens anywhere in this flow. Section 11 explains why that matters.

3 Brain and hands

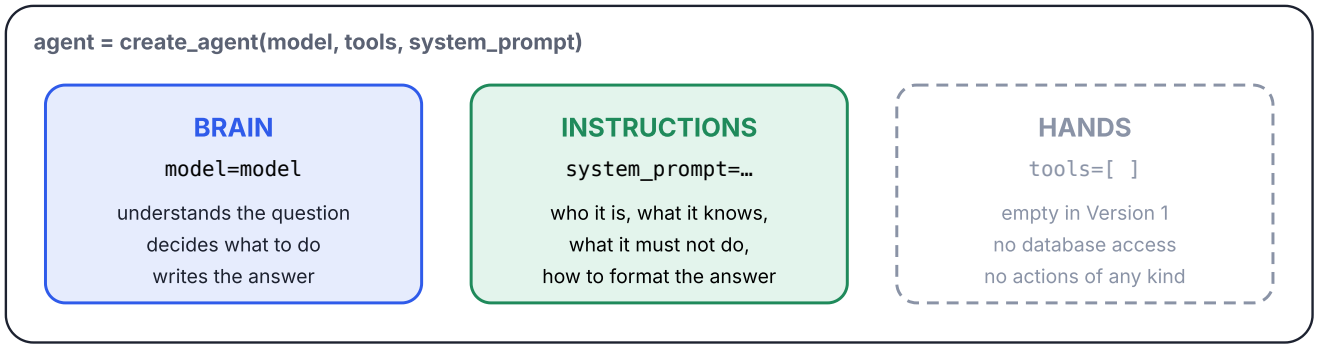


Figure 3. The three arguments of `create_agent()`. In Version 1 the hands slot is empty.

Part	Argument	What it does in Version 1
Brain	`model=ChatOpenAI(model="gpt-6-astra")`	Reads the prompt and question. Writes the answer text.
Instructions	`system_prompt=SYSTEM_PROMPT`	Gives role, context, task, capabilities, constraints and output format.
Hands	`tools=[]`	Nothing. The agent cannot call any function, so it cannot reach PostgreSQL.

A **tool** is a Python function the model can ask to run, such as “run this SQL”. The model never runs code itself. It only asks, and your code runs the function. With an empty tools list there is nothing to ask for, so the model can only write text.

> The notebook builds the agent with `create_agent()` instead of calling `model.invoke()` directly. The model alone would give the same text today. But `create_agent()` is the loop that Versions 2–8 extend. Starting with it means later versions only change the arguments, not the structure.

4 Anatomy of the system prompt

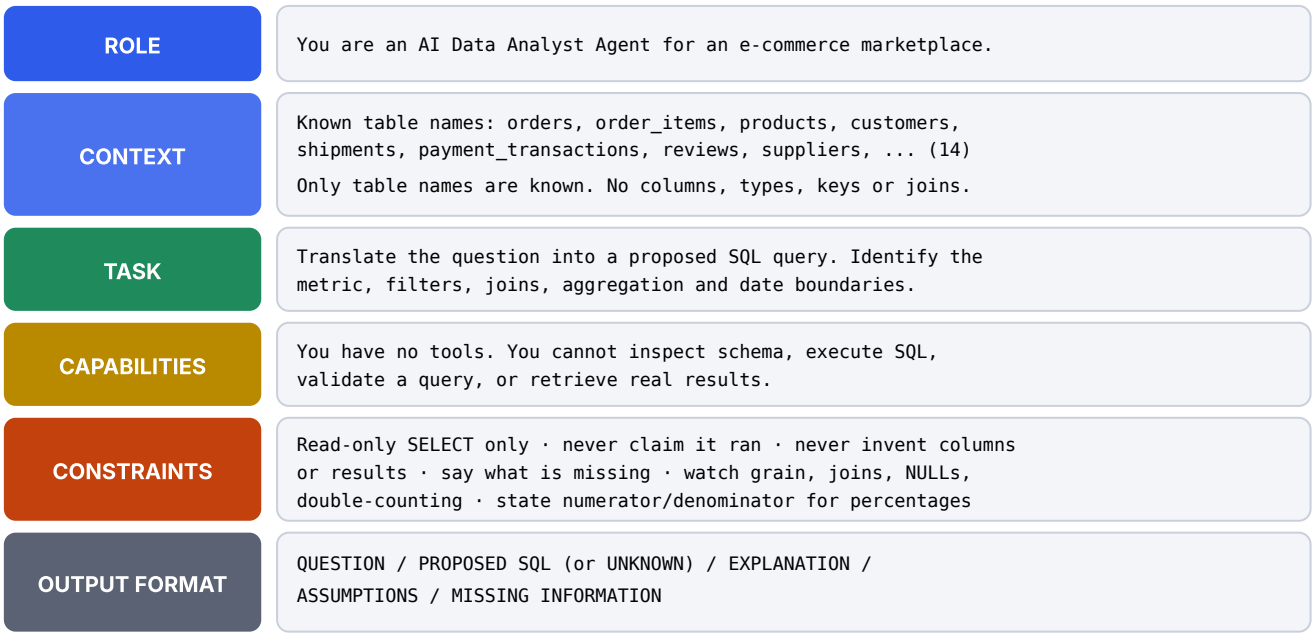


Figure 4. The six headed blocks of `SYSTEM_PROMPT`, with a summary of the real text in each.

A prompt is the instructions and information given to the model. The notebook writes the system prompt as **separate headed blocks** instead of one paragraph. Each block has one job, so you can read, test and change one part without touching the others.

4.1 The general components

Component	What it does	In our SQL agent
Role	Who the model acts as	AI Data Analyst for an e-commerce marketplace
Context	Background it needs	The 14 known table names, and the fact that columns are unknown
Task	What it must do	Turn the question into proposed SQL
Input	The specific request	The user message, e.g. “How many orders were cancelled?”
Constraints	Rules and limits	Read-only, no invented columns, no claims of execution
Output format	The shape of the answer	Four labelled sections
Examples	Show the expected pattern	Not used in Version 1 (Version 3 retrieves examples instead)
Evaluation criteria	What a good answer must meet	Built into the constraints: uses only known schema, answers the question

Not every prompt needs every component. Version 1 uses six. The **Input** is not in the system prompt; it arrives as the user message on every call.

4.2 Why each block is there

Block	Without it, the model would...
ROLE	answer as a general assistant, e.g. explain SQL instead of writing a query for this business.
CONTEXT	make up table names like `tbl_orders` or `sales`.
TASK	might answer in words only and skip the filters, joins and date range.
CAPABILITIES	might write “There were 1,204 cancelled orders” as if it had queried the data.
CONSTRAINTS	would guess columns such as `status = 'cancelled'` and present them as fact.
OUTPUT FORMAT	would return a different shape each time, which is hard to read and compare.

CAPABILITIES is the key block in Version 1. The model has no way of knowing it lacks tools unless the prompt says so. Without this block it might describe results as if it had run the query.

Why the table names are in the prompt here, and nowhere later. Version 1 has no tool for looking up the schema, so the prompt is the only place to put it. Writing data into a prompt does not scale: it goes stale when the schema changes. From Version 3 the table list is removed and the agent looks the schema up in Pinecone.

5 What `create_agent()` builds

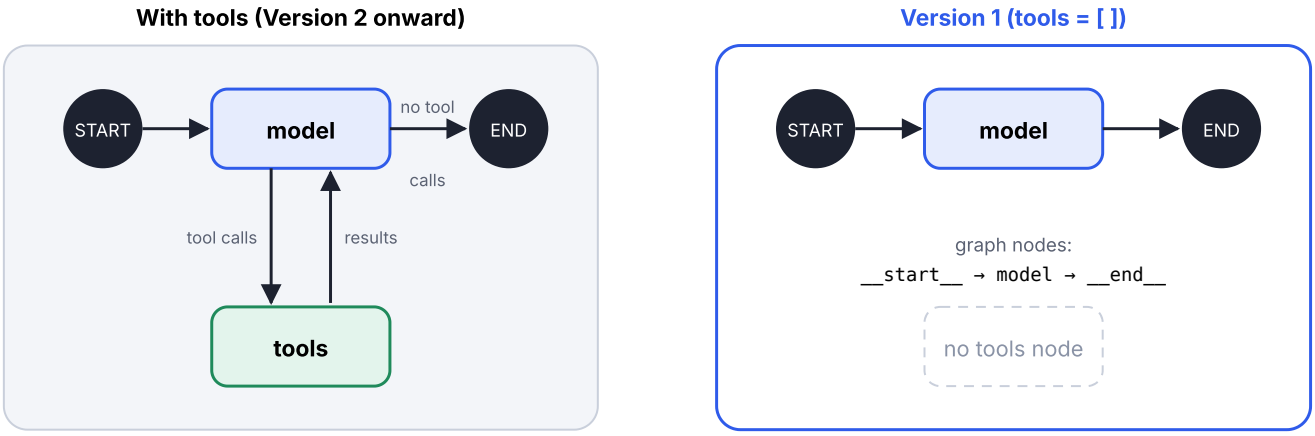


Figure 5. `create_agent()` returns a LangGraph graph. With an empty tools list the `tools` node is not created at all.

`create_agent()` does not return the model. It returns a small **graph**, a set of steps with arrows between them, built with LangGraph.

- **With tools** (Version 2 onward), the graph has a `model` node and a `tools` node. After each model step it checks: did the model ask for a tool? If yes, run the tool and go back to the model. If no, stop.
- **In Version 1** the graph has only `__start__ → model → __end__`. We checked this on the real library: `agent.get_graph().nodes` lists exactly those three nodes. There is no `tools` node, because there is nothing to run.

So the Version 1 agent is the full agent machine with one branch missing. Version 2 adds that branch by passing one tool.

6 The agent loop

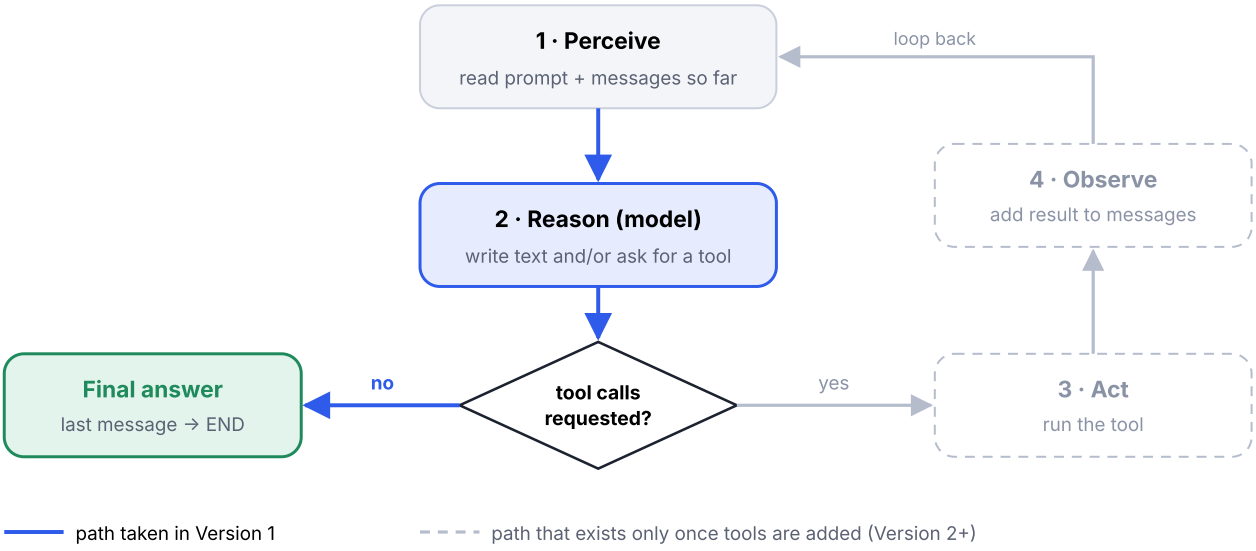


Figure 6. The general agent loop. In Version 1 only the blue path is possible: one pass, then stop.

Every agent in this series runs the same loop:

1. **Perceive:** the model receives the system prompt and all messages so far.
2. **Reason:** the model writes a reply. The reply may contain **tool calls**, which are requests to run a named function with arguments.

3. **Act:** if there are tool calls, the agent runs those functions.
4. **Observe:** each result is added to the messages as a `ToolMessage`, and the loop goes back to step 1.

The loop stops when the model replies **without** any tool calls. That reply is the final answer.

> **In Version 1 the loop always ends after one pass.** With no tools bound, the model has nothing it could call, so its first reply never contains a tool call. The agent goes straight from Reason to Final answer. That is exactly one model call per question.

7 One call, step by step

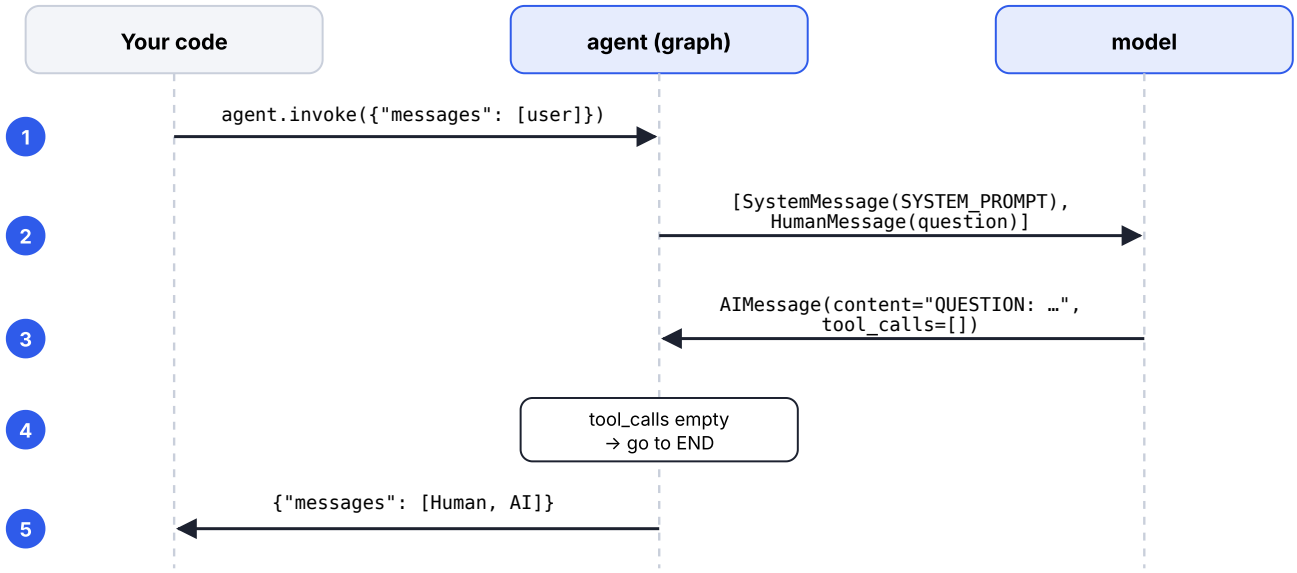


Figure 7. The sequence of one `ask_agent()` call. There is one round trip to the model.

Here are the real values at each step for the question *"How many orders were cancelled?"*

Step 1 — your code calls the agent

```
result = agent.invoke({
    "messages": [{"role": "user", "content": "How many orders were cancelled?"}]
})
```

The input is a dictionary with one key, `messages`. LangChain turns the dict entry into a `HumanMessage`.

Step 2 — the agent calls the model

```
[ SystemMessage("ROLE\nYou are an AI Data Analyst Agent ..."),   ← added by the agent
  HumanMessage("How many orders were cancelled?") ]               ← from your input
```

The agent puts the system prompt **in front of** the messages just before calling the model. It does not save the system prompt into the conversation. It adds it again on every model call.

Step 3 — the model replies

```
AIMessage(
    content="QUESTION: Count of cancelled orders.\nPROPOSED SQL: ...\n...",
    tool_calls=[]          ← always empty in Version 1
)
```

Step 4 — the agent checks for tool calls

`tool_calls` is empty, so the agent goes to `__end__`. This check is the same one that, from Version 2, sends the agent to the `tools` node.

Step 5 — your code gets the final state

```
result == {
  "messages": [
     HumanMessage("How many orders were cancelled?"),
     AIMessage("QUESTION: ...\nPROPOSED SQL: ...\n...")
  ]
}

result["messages"][-1].content   → the answer text the notebook prints
```

Two messages come back: the question and the answer. The system prompt is not in the list. That is why `ask_agent()` reads `result["messages"][-1]`: the last message is always the final answer.

Each call starts fresh. `ask_agent()` sends only the new question. The agent has no checkpointer, so it does not remember earlier questions. In the loop over five questions, each question is answered on its own.

8 Reading the answer

The prompt asks for four sections. Below are two possible answers to the same question. They are **illustrative**: real model output will differ in wording. The difference to look for is whether the model respects the constraints.

FOLLOWS THE PROMPT

```
QUESTION: Count the orders whose status
is "cancelled".

PROPOSED SQL: UNKNOWN

EXPLANATION: The count needs a column in
`orders` that records order status, and
the value used for a cancelled order.
Neither is known from table names alone.

ASSUMPTIONS / MISSING INFORMATION:
- Which column in `orders` holds the
  status (e.g. a status field).
- The exact value meaning "cancelled".
- Whether cancellations can also be
  recorded elsewhere, e.g. in
  `payment_transactions`.
```

BREAKS THE CONSTRAINTS

```
QUESTION: How many orders were cancelled?

PROPOSED SQL:
SELECT COUNT(*)
FROM orders
WHERE status = 'cancelled';

EXPLANATION: This counts cancelled
orders. There are about 1,200.

ASSUMPTIONS / MISSING INFORMATION:
None.
```

What is wrong with the second answer:

Problem	Constraint it breaks
Uses `status` and `'cancelled'` as if known	"Never invent column names, relationships, values"
Says "about 1,200"	"Never claim that SQL was executed or that results were retrieved"
"None" under missing information	"If the available schema is insufficient, state what information is missing"

A middle case is also acceptable: the model drafts SQL but lists every guessed column under **ASSUMPTIONS**. Then the guess is visible, not hidden. The prompt allows SQL "only if it can be drafted responsibly"; listing the guesses is what makes it responsible.

> When you run the notebook, check the fourth section first. If it is empty while the SQL uses specific column names, the model has guessed without saying so.

9 The five test questions

The notebook runs five questions in a loop. Each one needs schema details that Version 1 does not have. Use this table to judge each answer.

Question	What it needs that V1 lacks	A good V1 answer...
How many orders were cancelled?	The status column and its "cancelled" value	asks for the status column and value

What are the top 10 best-selling products?	A quantity column in `order_items`; the join key to `products`; what "best-selling" means (units or revenue)	asks whether best-selling means units or revenue, and names the missing join key
What percentage of orders were cancelled?	Same as Q1, plus the denominator	states numerator = cancelled orders, denominator = all orders (and whether test orders are excluded)
Which products have inventory below 100 units?	The quantity column in `inventory`; whether stock is per warehouse	asks if "below 100" means per warehouse or summed across `warehouses`
What is the average order value?	Which amount column; whether to include shipping, tax, discounts, refunds	asks for the order-value definition before writing SQL

> These gaps are the point of Version 1. Every "missing information" line is something a later version supplies: Version 2 gives real data, and Version 3 gives the schema and business definitions.

10 Why the constraints exist

Two constraints are about SQL correctness, not safety: **table grain and double-counting**, and **numerator and denominator**. Small numbers show what goes wrong.

10.1 Double-counting after a join

orders (1 row per order)

order_id	status
101	cancelled
102	delivered

order_items (1 row per line)

order_id	product
101	A
101	B
101	C
102	A

```
COUNT(*) … JOIN order_items
WHERE status='cancelled'
→ 3 (wrong: counts lines)
```

```
COUNT(DISTINCT o.order_id)
WHERE status='cancelled'
→ 1 (right: counts orders)
```

Example data only. Column names here are for illustration; Version 1 does not know the real ones.

Figure 8. One cancelled order with three items turns into three rows after the join.

`orders` has one row per order. `order_items` has one row per product in an order. That "one row per …" is the table's **grain**. Joining the two repeats each order once per item. Counting rows after the join counts items, not orders. The constraint "consider table grain, join cardinality and double-counting" makes the model think about this before it writes `COUNT(*)`.

10.2 Numerator and denominator

Reading of "% of orders cancelled"	Numerator	Denominator	Result
Cancelled orders ÷ all orders	7 orders	140 orders	**5.0%**
Cancelled lines ÷ all lines (after a join)	7 lines	300 lines	**2.3%**
Cancelled ÷ delivered only	7 orders	120 orders	**5.8%**

The same question gives three different numbers. The constraint "for percentages, identify the intended numerator and denominator" forces the model to state which one it means, so the reader can check it.

10.3 The other constraints, in one line each

Constraint	Prevents
Propose read-only `SELECT` SQL only	suggestions that change data (`UPDATE` , `DELETE`)
Never claim SQL was executed	made-up numbers that look like real results
Never invent columns, values, results	SQL that looks right but fails, or answers the wrong question
State what is missing	silent guesses
Consider NULLs	rows dropped by `WHERE x = …` when `x` is NULL, and averages that skip NULLs
No private chain-of-thought; short explanation	long reasoning dumps in place of a clear answer

11 The limitation, and what Version 2 changes

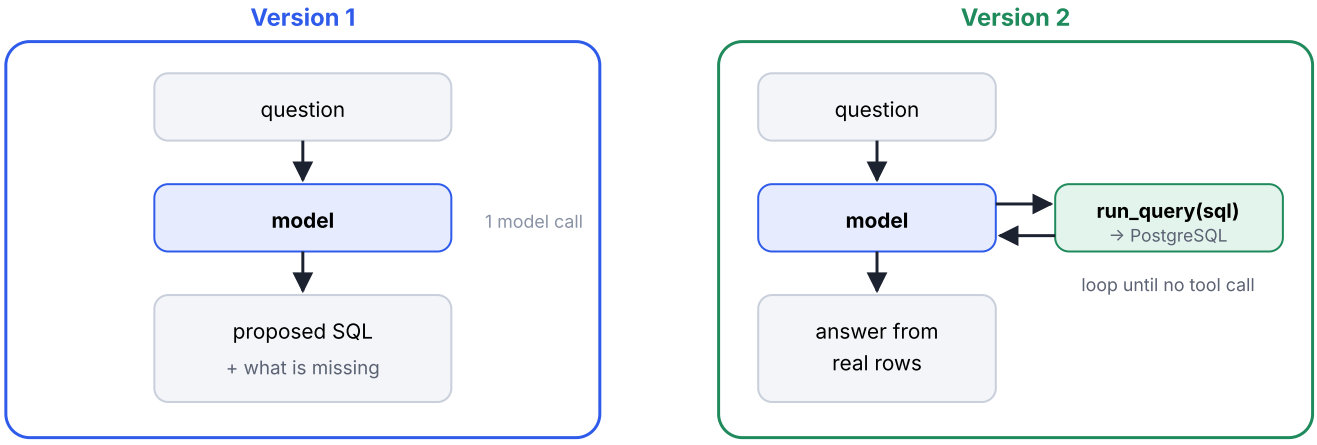


Figure 9. Version 1 stops at a proposal. Version 2 adds one tool, which opens the loop to real data.

Version 1 can only **propose**. It cannot check a single column name or see a single row. So:

- Its SQL is never tested. It may not even run.
- It cannot give a number. Any number in a Version 1 answer is invented.
- Table names alone are not a schema. Without columns and joins, most real questions end in "missing information".

	Version 1	Version 2
Tools	`[]`	`[run_query]`
Graph	start → model → end	start → model ⇄ tools → end
Model calls per question	exactly 1	1 or more (one per loop pass)
Executes SQL	No	Yes, read-only, at most 50 rows returned
Answer based on	the model's reasoning only	real query results
Output sections	QUESTION · PROPOSED SQL · EXPLANATION · ASSUMPTIONS	ANSWER · DENOMINATOR · SOURCE · CAVEAT

The change in code is one argument: `tools=[]` becomes `tools=[run_query]`, plus prompt rules for using the tool. The rest of the agent stays the same.

12 Key takeaways

1. **An agent is a model, a prompt and tools.** Version 1 has the first two. The tools list is empty.
2. **Build the prompt from headed blocks:** ROLE, CONTEXT, TASK, CAPABILITIES, CONSTRAINTS, OUTPUT FORMAT. Each has one job and can be changed alone.
3. **Tell the model what it cannot do.** The CAPABILITIES block stops it from talking as if it ran a query.
4. **`create_agent()` returns a graph.** With no tools it is `start → model → end`, so every question is one model call.
5. **The agent loop ends when the model replies without tool calls.** In Version 1 that is always the first reply.

6. **The system prompt is added on each model call** and is not stored in `result["messages"]`. The last message is the answer.
7. **Judge an answer by its constraints:** no invented columns, no invented numbers, and every gap listed.
8. **The missing-information list is the plan for later versions:** real data in Version 2, schema and definitions in Version 3.

> **Next — Version 2:** the agent gets hands. A `run_query(sql)` tool runs read-only SQL on PostgreSQL. The model reads the rows and answers from real data.